

# Belief-Driven Planning Architecture

## End-to-End Example Run (Conference Planning Demo)

---

### Scenario

User Request:

"Plan a 3-day AI conference in Manchester for 500 attendees with a budget of £50,000."

---

### Step 1: Load Permanent Beliefs

File:

permanent\_beliefs.json

```
{
  "beliefs": [
    {
      "id": "PB001",
      "type": "agent_capability",
      "statement": "VenueAgent can search and evaluate conference venues."
    },
    {
      "id": "PB002",
      "type": "agent_capability",
      "statement": "SpeakerAgent can identify and evaluate conference speakers."
    },
    {
      "id": "PB003",
      "type": "agent_capability",
      "statement": "BudgetAgent can calculate and validate conference budgets."
    },
    {
      "id": "PB004",
      "type": "agent_capability",
      "statement": "ScheduleAgent can create conference schedules."
    }
  ]
}
```

---

## Step 2: Create Empty Instance Belief Base

File:

instance\_beliefs.json

```
{
  "task_id": "CONF001",
  "beliefs": []
}
```

## Step 3: User Goal Arrives

Input:

Plan a 3-day AI conference in Manchester  
for 500 attendees  
with a budget of £50,000.

## Step 4: Goal Interpreter (LLM Layer)

Input:

Natural Language Goal

Output:

goal.json

```
{
  "goal_id": "GOAL001",
  "goal_name": "conference_ready",

  "constraints": {
    "location": "Manchester",
    "attendees": 500,
    "budget": 50000,
    "duration_days": 3
  },

  "required_outcomes": [
    "budget_approved",
```

```
    "venue_booked",  
    "speakers_confirmed",  
    "schedule_created",  
    "conference_ready"  
  ]  
}
```

---

## Step 5: Planner Execution

Planner Input

Goal:

conference\_ready

---

Backward Chaining Rules

```
{  
  "conference_ready": [  
    "schedule_created"  
  ],  
  "schedule_created": [  
    "speakers_confirmed"  
  ],  
  "speakers_confirmed": [  
    "venue_booked"  
  ],  
  "venue_booked": [  
    "budget_approved"  
  ]  
}
```

---

Planner Output

execution\_plan.json

```
{  
  "plan": [  
    "budget_approved",  
    "venue_booked",  
  ]  
}
```

```
    "speakers_confirmed",  
    "schedule_created",  
    "conference_ready"  
  ]  
}
```

---

## Step 6: Intention Assignment

Goal:

budget\_approved

Available Agents:

```
[  
  {  
    "agent": "BudgetAgent",  
    "capabilities": [  
      "calculate_budget"  
    ]  
  },  
  {  
    "agent": "VenueAgent",  
    "capabilities": [  
      "search_venue"  
    ]  
  }  
]
```

---

LLM Selection Output

```
{  
  "goal": "budget_approved",  
  "assigned_agent": "BudgetAgent",  
  "confidence": 0.99  
}
```

---

## Step 7: Agent Execution

BudgetAgent executes.

Mock Result:

```
{
  "status": "success",
  "remaining_budget": 50000
}
```

---

## Step 8: Belief Update

New Belief Generated

```
{
  "id": "IB001",
  "type": "task_fact",
  "statement": "Budget approved for £50,000.",
  "source": "BudgetAgent",
  "confidence": 0.99
}
```

Updated Instance Beliefs

```
{
  "task_id": "CONF001",

  "beliefs": [
    {
      "id": "IB001",
      "statement": "Budget approved for £50,000."
    }
  ]
}
```

---

## Step 9: Next Goal

Planner marks:

budget\_approved = complete

Next Goal:

venue\_booked

---

## Step 10: Venue Agent Execution

Assigned:

VenueAgent

Mock Result:

```
{
  "venue_name": "Manchester Central",
  "estimated_cost": 18000
}
```

---

Belief Update

```
{
  "id": "IB002",
  "type": "task_fact",
  "statement": "Manchester Central venue selected for £18,000.",
  "source": "VenueAgent"
}
```

## Step 11: Speaker Agent Execution

Goal:

speakers\_confirmed

Output:

```
{
  "speakers": [
    "Speaker A",
    "Speaker B",
    "Speaker C"
  ]
}
```

Belief Update

```
{
  "id": "IB003",
  "statement": "Three conference speakers confirmed."
}
```

---

## Step 12: Schedule Agent Execution

Goal:

schedule\_created

Output:

```
{
  "days": 3,
  "sessions": 24
}
```

Belief Update

```
{
  "id": "IB004",
  "statement": "Conference schedule created."
}
```

---

## Step 13: Goal Completion Check

Planner verifies:

```
budget_approved ✓
venue_booked ✓
speakers_confirmed ✓
schedule_created ✓
```

---

Generated Belief

```
{
  "id": "IB005",
  "statement": "Conference planning completed successfully."
}
```

---

## Final Instance Belief Base

```
{
  "task_id": "CONF001",

  "beliefs": [
    {
      "id": "IB001",
      "statement": "Budget approved for £50,000."
    },
    {
      "id": "IB002",
      "statement": "Manchester Central venue selected."
    },
    {
      "id": "IB003",
      "statement": "Three conference speakers confirmed."
    },
    {
      "id": "IB004",
      "statement": "Conference schedule created."
    },
    {
      "id": "IB005",
      "statement": "Conference planning completed successfully."
    }
  ]
}
```

---

## Failure Scenario Example

Goal:

venue\_booked

VenueAgent returns:



```
{
  "venue_name": "Premium Hall",
  "cost": 70000
}
```

---

Belief Generated

```
{
  "statement": "Venue cost exceeds budget."
}
```

---

Planner Evaluation

Current State:

```
Budget = £50,000
Venue Cost = £70,000
```

Goal:

venue\_booked

Status:

FAILED

---

Automatic Replanning

New Goal Generated

```
{
  "goal": "find_alternative_venue"
}
```

New Execution Plan

```
{
  "plan": [
    "find_alternative_venue",
    "venue_booked",
  ]
}
```

```
    "speakers_confirmed",  
    "schedule_created",  
    "conference_ready"  
]  
}
```

---

## Prototype Folder Structure

```
project/  
  
├─ beliefs/  
│   ├── permanent_beliefs.json  
│   └── instance_beliefs.json  
  
├─ planner/  
│   ├── goal_interpreter.py  
│   ├── backward_chaining.py  
│   └── planner.py  
  
├─ agents/  
│   ├── venue_agent.py  
│   ├── speaker_agent.py  
│   ├── budget_agent.py  
│   └── schedule_agent.py  
  
├─ intentions/  
│   └── intention_manager.py  
  
├─ knowledge/  
│   └── belief_manager.py  
  
├─ demos/  
│   └── conference_demo.py  
  
└─ tests/
```

---

## Success Path

User Goal ↓ Goal Interpreter ↓ Structured Goal ↓ Backward Chaining Planner ↓ Execution Plan ↓  
Agent Assignment ↓ Execution ↓ Belief Updates ↓ Goal Verification ↓ Conference Ready

This document acts as the canonical execution flow for Version 1 of the architecture.